目录

第一部分 “算法实现”设计说明 3

1.1 题目 3

1.2 设计思想 3

1.3 逻辑结构与物理结构 3

1.4 开发平台 3

1.5 软件功能 4

1.6 系统的运行结果分析说明 5

1.7 安装、运行与操作 14

第二部分 “综合应用”设计说明 14

2.1 题目 14

2.2 设计思想 15

2.3 逻辑结构与物理结构 15

2.4 开发平台 15

2.5 软件功能 15

2.6 系统的运行结果分析说明 17

2.7 安装、运行与操作 21

第三部分 总结与收获 21

第四部分 学习资料 22

第一部分 “算法实现”设计说明

1.1 题目

- 链式线性表操作的可视化

分别实现基于单链表、循环链表和双向链表的线性表基本操作，要求实时展示操作过程中的数据结构变化。基本操作包括:

(1)建立:通过输入序列创建链表;

(2)插入:在指定位置插入新元素:

(3)删除:删除指定位置的元素;

(4)查找:按值查找元素位置。

1.2 设计思想

本次软件开发在 Qt Creator 中完成，根据 Qt 开发的基本思想，本次软件开发的设计思想也主要分为两部分，即界面开发和逻辑功能开发。经过自主学习和探究，使用 Qt 进行开发的过程主要就是先进行可视化界面的开发设计，再通过 Qt 开发中的“信号与槽”机制，将可视化界面上的操作接入后端的逻辑功能。

在具体的设计开发过程中，本着“实现三种链表--单链表、循环链表、双向链表的基本操作，用可视化界面实时展示链表结构变化”的设计目标，规划用户交互界面的设计；通过“信号与槽”机制，提供交互界面和逻辑功能的接口；采用面向对象的编程方式，分别编写数据结构实现模块、操作实现模块、动画可视化模块等，分类实现各个操作。

1.3 逻辑结构与物理结构

本软件基于线性表这一基本数据结构类型和四个基本操作进行开发设计。线性表是一种线性数据结构，各个节点之间通过指针域互相连接，在实际的内存存储空间中并非连续的物理存储，只是逻辑上的连续。

三种链表的逻辑结构有略微不同：单链表每个节点只包含指向下一个节点的指针域，无法找到其上一个节点；循环链表在单链表的基础上略有不同，单链表尾节点指向 null，但循环链表的尾节点指回头节点，实现循环结构；双向链表每个节点包含“前驱”和“后继”两个指针域，支持双向遍历。

三种链表的物理结构的实现依赖于不同编程语言的内存管理机制。本次使用 C++，则主要是通过 C++的内存管理机制 new/delete 进行链表节点的物理结构的实现。具体实现时，每个链表节点用一个结构体表示，结构体内主要的组成为数据域--int 型，一个前驱指针，一个后继指针。为了设计的简便，三种链表的节点结构是统一的，都有前驱和后继，只是会结合具体的链表类型选取后继或前驱和后继的组合。节点在实际的内存种存储并不连续，通过指针连接成一个逻辑上的线性结构。

1.4 开发平台

本次课程设计具体的开发平台为 Windows11 的操作系统 + Qt Creator 17.0.0(Community)的集成开发环境 + C++的编程语言(C++11 标准以上)。具体生成可执行程序的编译器为 MinGW 13.1.0 64-bit，Qt 版本 6.9.1。

1.5 软件功能

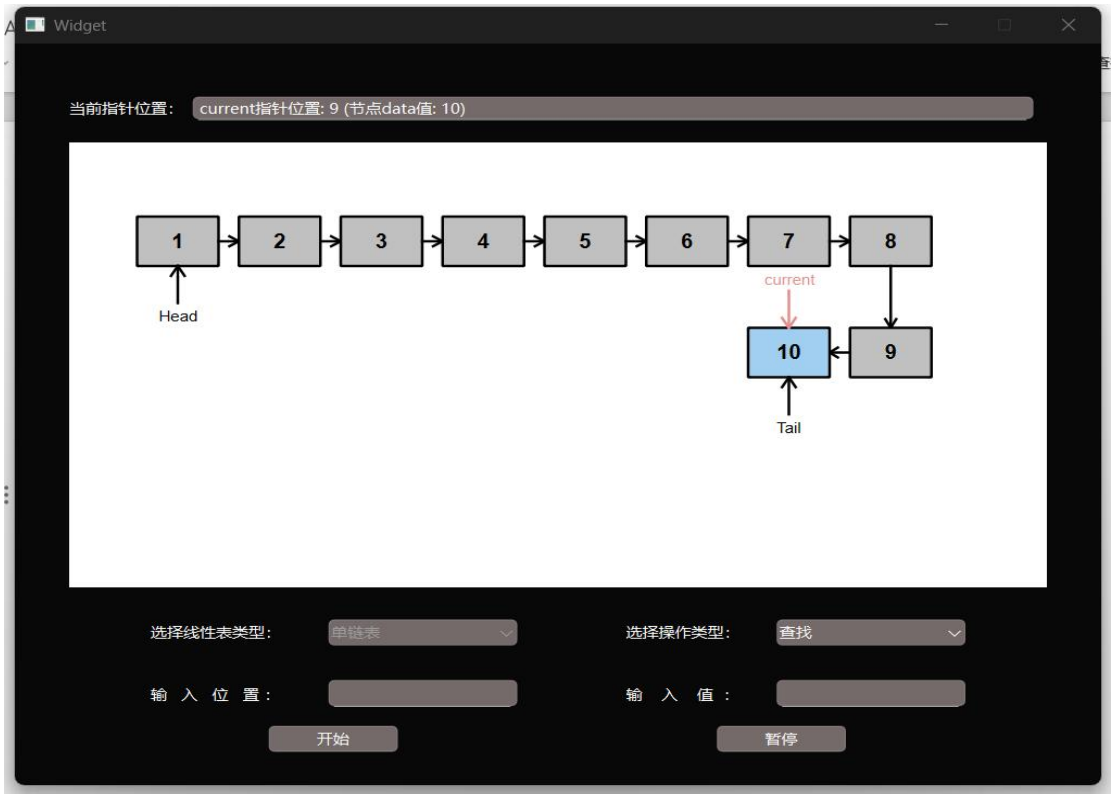

图 1：软件系统界面

打开软件后的效果如上图所示。每个控件的具体功能为：

“当前指针位置”：会实时显示 current 指针所指向的节点的位置。

“选择线性表类型”：提供**单链表**、**双向链表**、**循环链表**三种链表类型，可根据需求进行选择。

“选择操作类型”：提供**建立**、**查找**、**插入**、删除四种链表操作类型，可根据需求进行选择。

“输入位置”：在**删除**、**插入**操作时会用到，一次仅支持一个数字的输入。不符合要求的输入会有弹窗提示。

“输入值”：在**建立**、**插入**、**查找**操作时会用到。

- Head、tail、current 三个指针对链表进行标记（循环链表无 tail 指针）。
- 等待下一步操作时，current 所指的节点会闪烁（作为标记）。
- 选定操作后，按“开始”键会开始对应操作的动画演示，动画演示时按“暂停”键会暂停当前进行的动画演示，同时按键出现“继续”字样，再次点击即可继续当前动画演示。

1. **建立**：当没有链表建立时，必须先进行“建立”操作，才可继续选择其他操作（已设置错误操作的弹窗提示，并且一旦建立链表后，“选择线性表类型”的选择框将被锁定，无法再进行链表类型的更改）。“建立”选定类型链表后，必须将当前链表完全删除后才可“建立”新的链表。“建立”操作在“输入值”框中要输入建立链表的序列，链表节点的 data 值为 int 类型，在输入框中可用任意非数字的字符间隔不同节点的 data 值。当前设置了最大产生节点数为 16，输入超过 16 个数字（节点值），将会有错误弹窗。Current 指针会随着新产生的节点移动，“建立”操作进行完后，将会停留在 tail 节点处。

推荐输入格式：（用空格分割）0 1 2 3 4 5 6 7 8 9 10

弹窗错误处理机制：

- 输入值处为空时："建立链表的输入序列不能为空！"
- 输入不含任何一个数字是："输入中必需包含数字!"
- 输入的数字个数超过了最大节点数的限制："节点数量不能超过 16 个！当前输入了 x 个节点，请重新输入。"
- 成功建立链表后："已成功建立链表！删除完本链表后才能再次改变链表类型！"

2. 查找：链表建立后，可进行“查找”操作。仅支持一次输入一个数值进行查找。Current 指针会从头指针开始遍历整个链表，全部遍历完成后会跳出弹窗显示所输入数值在链表中的位置（在链表中重复出现的值，会显示所有出现的位置；链表中未出现的值，也会在遍历后显示“未找到指定元素值 xx”的弹窗提示）。已设置错误输入处理机制，在“输入值”一栏中输入的任何非数字字符都会被处理掉，只保留出现的数字值进行链表中的“查找”操作。
推荐输入格式：14（等纯数字）

- 当在“输入位置”处有输入时："请指定查找元素值，无需指定位置！"
- 输入多个查找值时："一次只能查找一个指定元素值！"
- “输入值”处没有任何一个数字时："请输入有效的数字！"

3. 删除：链表建立后，可进行“删除”操作。仅支持一次输入一个指定位置进行删除。已设置错误输入处理机制，在“输入位置”一栏中输入的任何非数字字符都会被处理掉，只保留出现的数字值进行链表中的“删除”操作，并且设置了各种错误数字处理机制，不合法的输入均无效。Current 指针会从头指针开始遍历整个链表，遍历到指定位置时，会直接删除当前位置所在的节点。整个链表删除完后，可以建立新的链表。

- “输入位置”一栏为空时："删除的位置不能为空！"
- “输入位置”一栏中有多个数字时："位置只能为 1 个数字！"
- “输入位置”一栏中含有其他字符时："输入的位置不能包含字符！"
- “输入位置”超过链表长度时："位置不合法！请重新输入！"
- 当在“输入值”一栏有输入时："请指定删除位置，无需指定值！"

4. 插入：链表建立后，可进行“插入”操作。仅支持一次输入一个指定位置和想要插入的节点值进行插入。已设置错误输入处理机制，在“输入位置”“输入值”一栏中输入的任何非数字字符都会被处理掉，只保留出现的数字值进行链表中的“插入”操作，并且设置了各种错误数字处理机制，不合法的输入均无效。Current 指针会从头指针开始遍历整个链表，遍历到指定位置时，会采用“头插法”进行插入，current 指针指向新插入的节点。

- “输入值”一栏不止一个值时："一次必须只能插入一个值！"
- “输入值”一栏有其他字符时："请输入有效的数字值！"
- “输入位置”一栏的值超过链表当前长度时："请输入有效的插入位置！"
- 链表节点超过最大节点数量限制时："链表节点数量已达上限（16 个），无法插入新节点！"

1.6 系统的运行结果分析说明

本系统在 Windows 11 操作系统，Qt 6.9.1 平台下运行，通过 MinGW 13.1.0 编译，通过 Qt Creator 17.0.0(Community)进行调试。实际运行过程中，用户可成功完成双向链表、循环链表、单链表的建立、插入、查找、删除等操作。系统界面如图 1 所示。

接下来，将通过多组正常的测试数据，系统地展示软件各功能的运行过程及结果分析说明。（各种错误输入及软件的错误响应过程在此不再演示，可由使用者自行测试。本软件邀

请了 5 位朋友进行功能测试，基本处理了所有可能出现的错误输入情况。）

测试数据一：单链表 建立 “1 2 3 4 5 6”

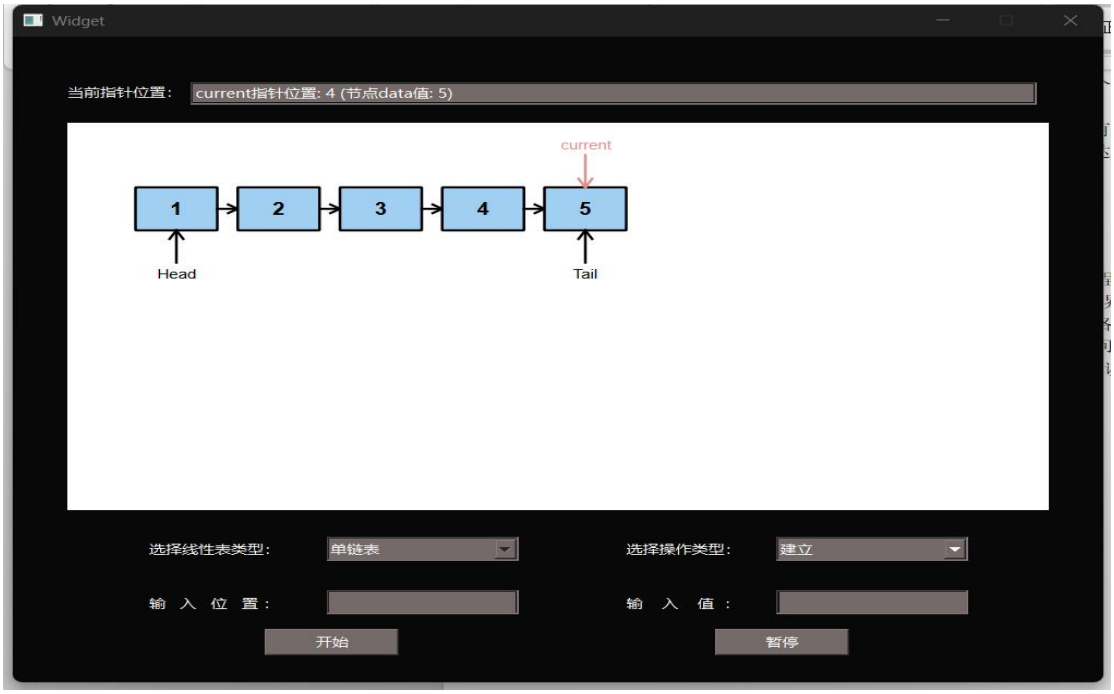

图 2：建立过程

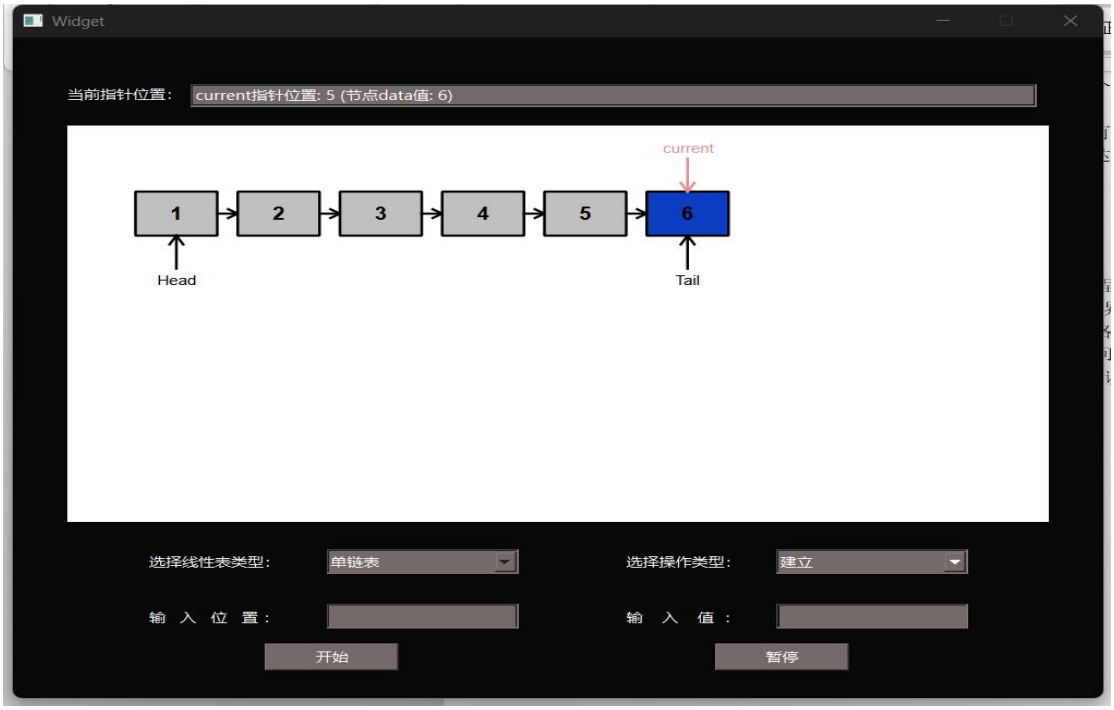

图 3：建立完成

点击“开始”，屏幕上会出现建立的动画演示过程（一个一个的节点逐渐建立），同时 current 指针会随着新建立的节点移动。建立完成后，current 指针所在的节点会闪烁（作为标记）。建立过程中点击“暂停”即可暂停当前动画演示，再次点击即可继续当前动画演示。

测试数据二：单链表 删除 “4”（删除输入位置为 4 的节点）

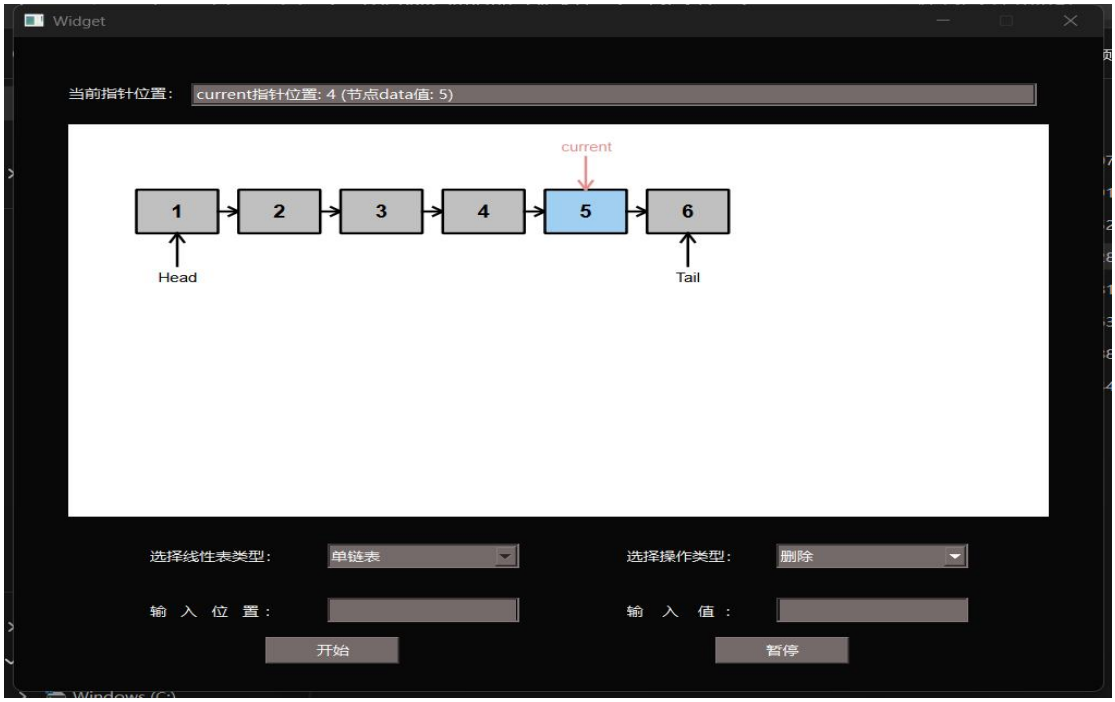

图 4：查找删除位置过程

选择好操作类型后，在输入位置处输入 4（位置从 0 开始编号），点击开始。Current 指针会从头节点开始遍历寻找，找到指定位置后，对该位置节点进行删除。（结果如图 5 所示）（此时链表已经建立，当前链表没有删除完的时候，“选择线性表类型”那一栏是没办法使用的。直到当前链表全部删除完，才能再次“选择线性表类型”进行新链表的建立。）

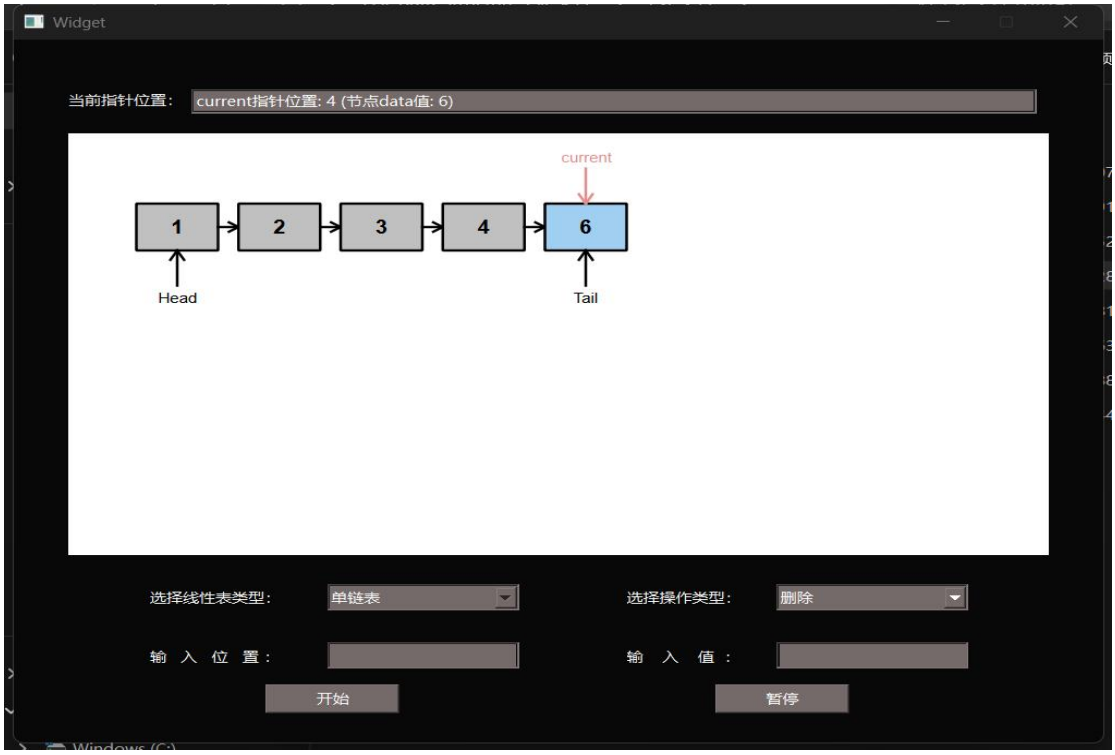

图 5：删除完成

测试数据三：单链表 插入 “2” “4”（在位置 2 处插入值为 4 的节点）

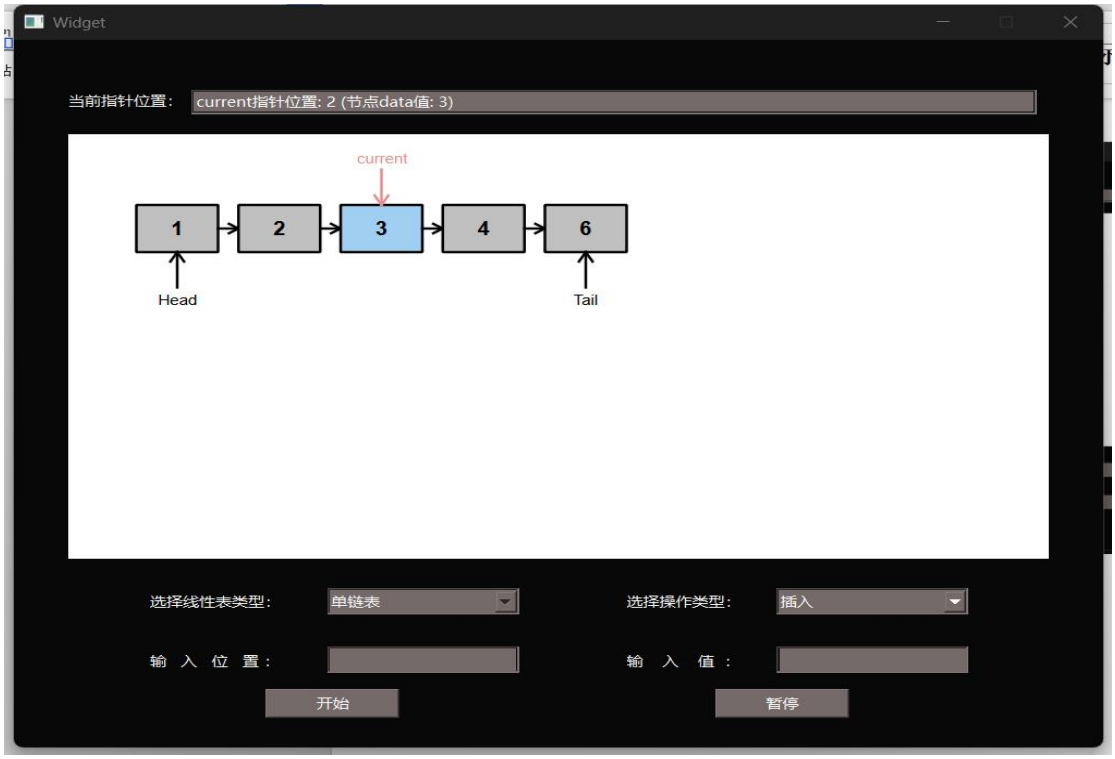

图 6：查找插入位置过程

选择好操作类型后，在输入位置处输入 2，在输入值处输入 4，点击开始。Current 指针会从头节点开始遍历，寻找插入的位置，找到插入位置（位置从 0 开始编号）后，在当前位置插入节点，最终结果如图 7 所示。并且插入完成后，current 指针所在的节点会闪烁（作为标记）。插入过程中点击“暂停”即可暂停当前动画演示，再次点击即可继续当前动画演示。

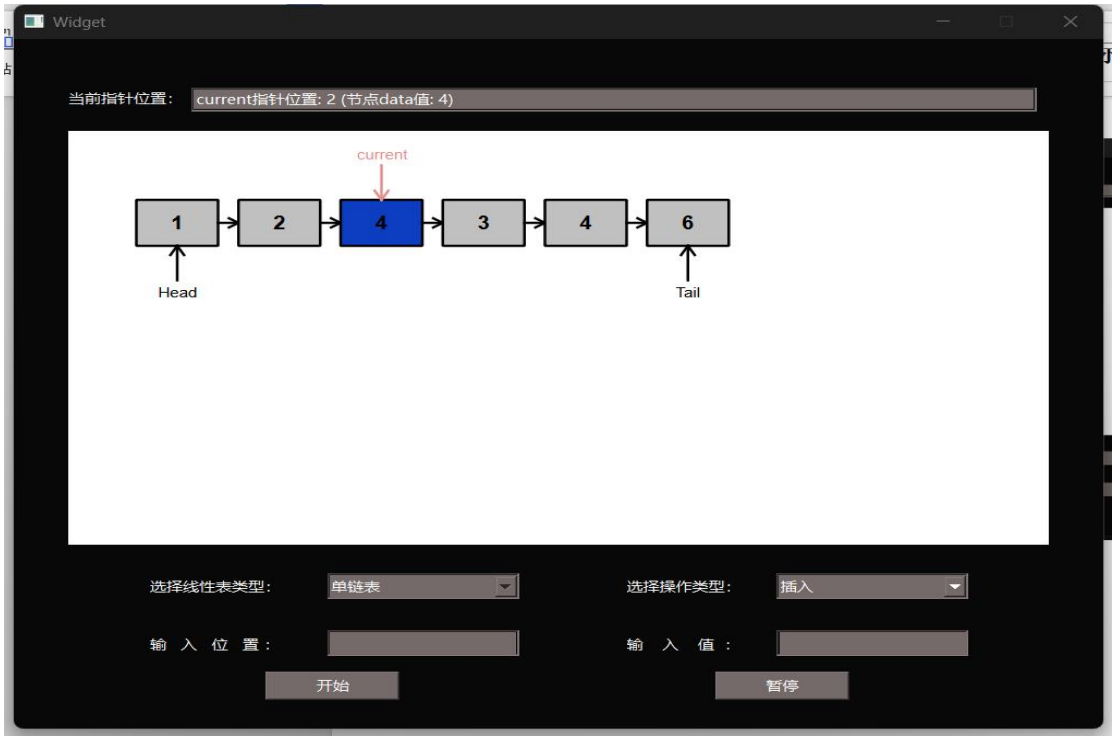

图 7：插入完成

测试数据四：单链表 查找 “4”（查找元素值为 4 的节点位置）

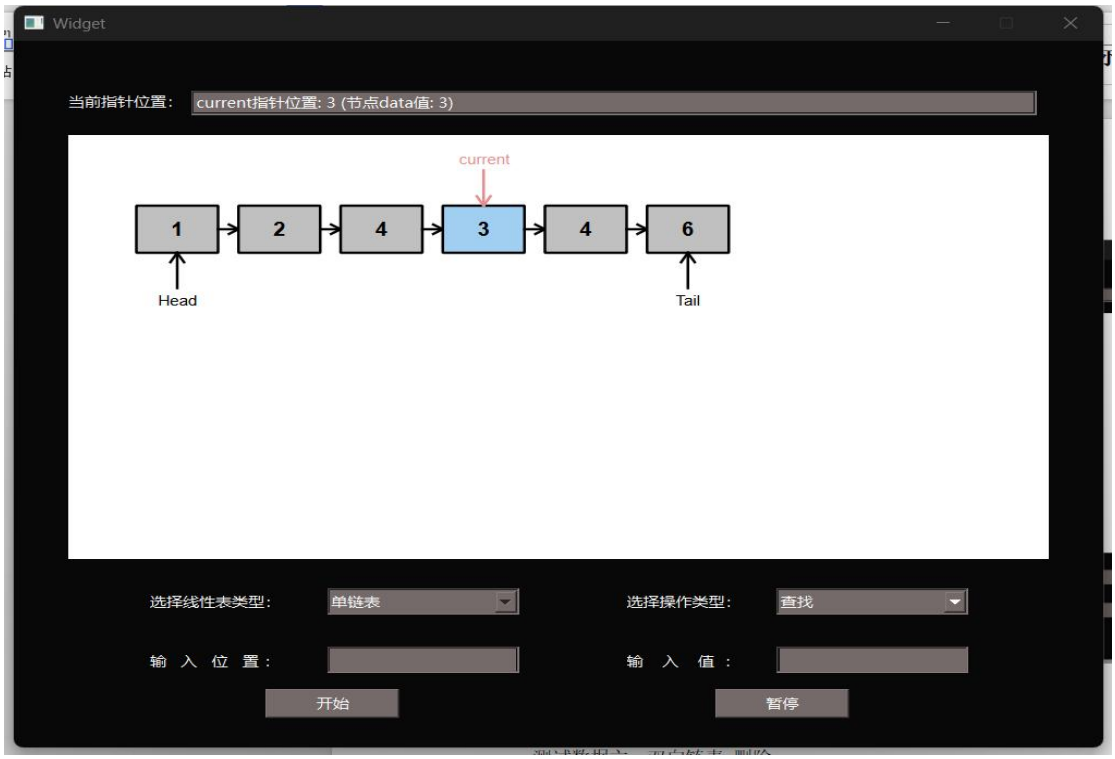

图 8：查找过程

选择好操作类型后，在输入值处输入 4，点击开始。Current 指针会从头节点开始遍历，寻找输入值为 4 的节点。遍历到尾节点后，会弹窗显示值为 4 的节点的所有位置（位置从 0 开始编号）。最终结果如图 9 所示。查找过程中点击“暂停”即可暂停当前动画演示，再次点击即可继续当前动画演示。

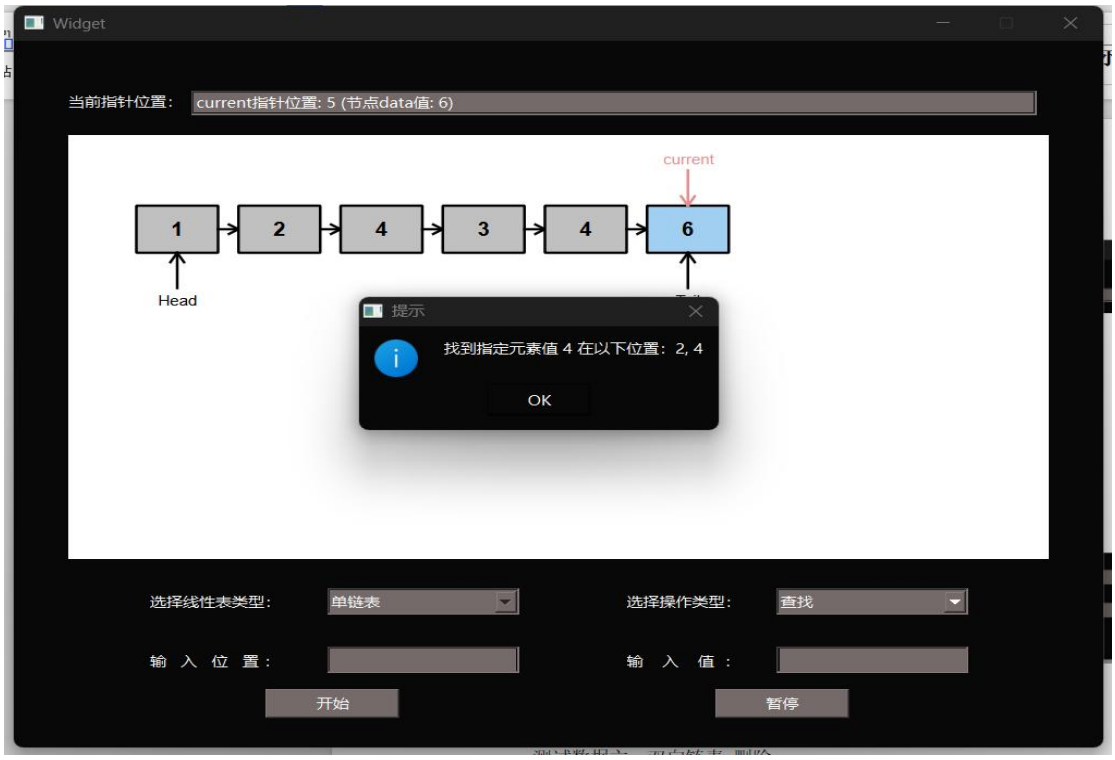

图 9：查找结果

接下来双向链表、循环列表的建立、插入、删除、查找操作与单链表的操作过程类似，故不再逐个说明，直接展示操作结果。

测试数据五：双向链表 建立 “1 2 3 4 5 6 7 8”

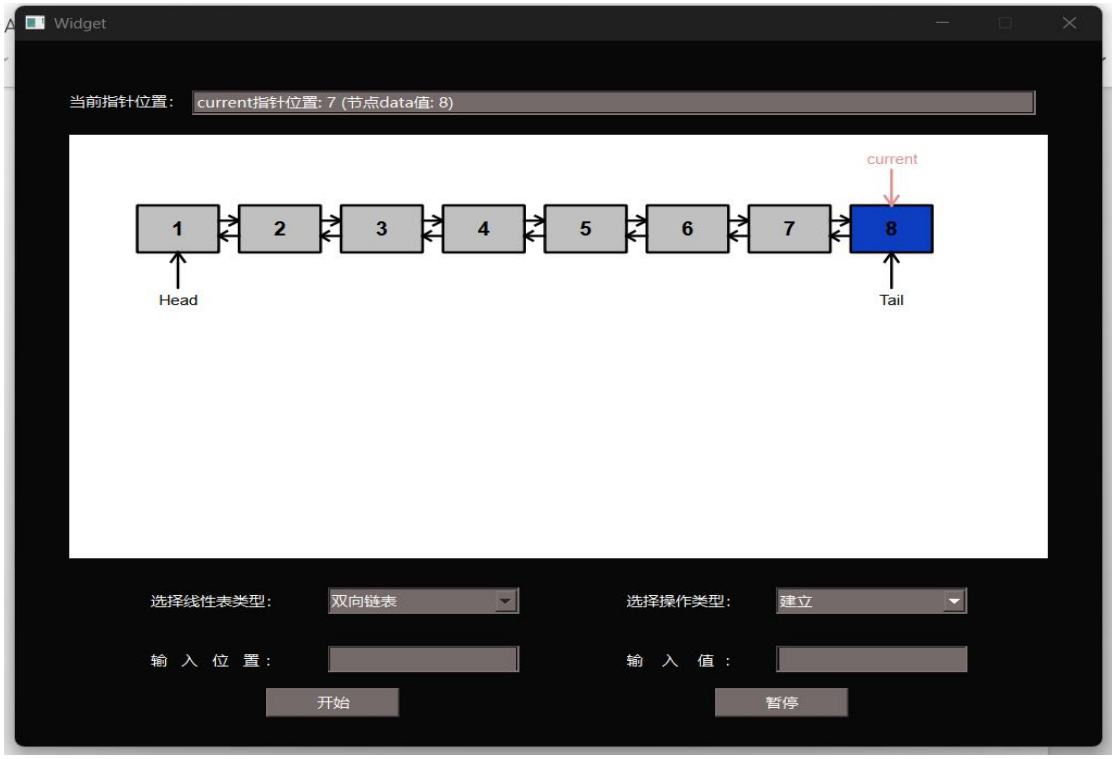

图 10：建立结果

测试数据六：双向链表 删除 “4”

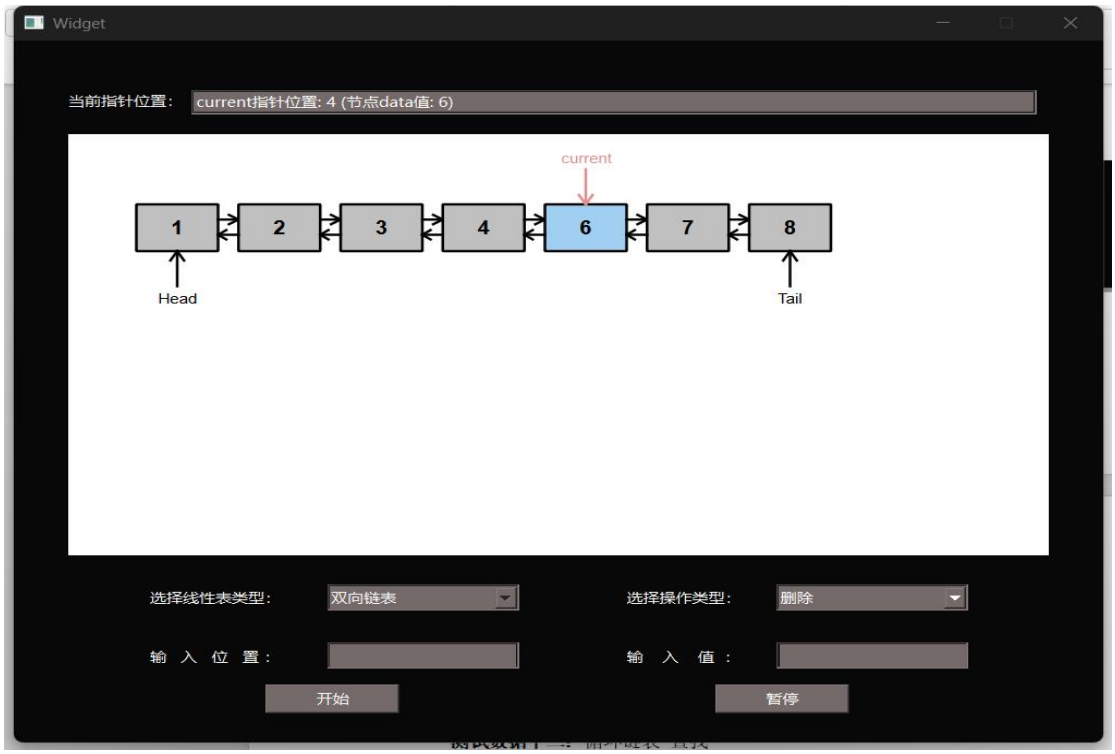

图 11：删除结果

测试数据七：双向链表 查找 “9”

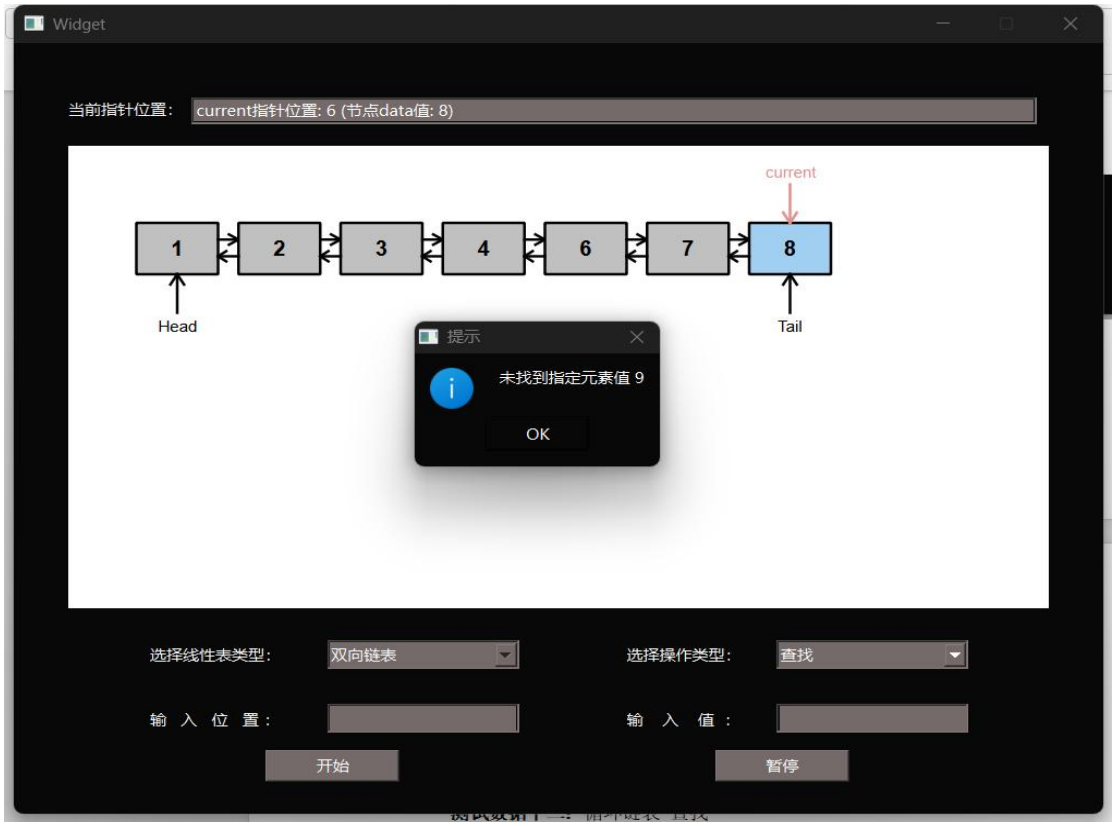

图 12：查找结果

测试数据八：双向链表 插入 “2” “88”

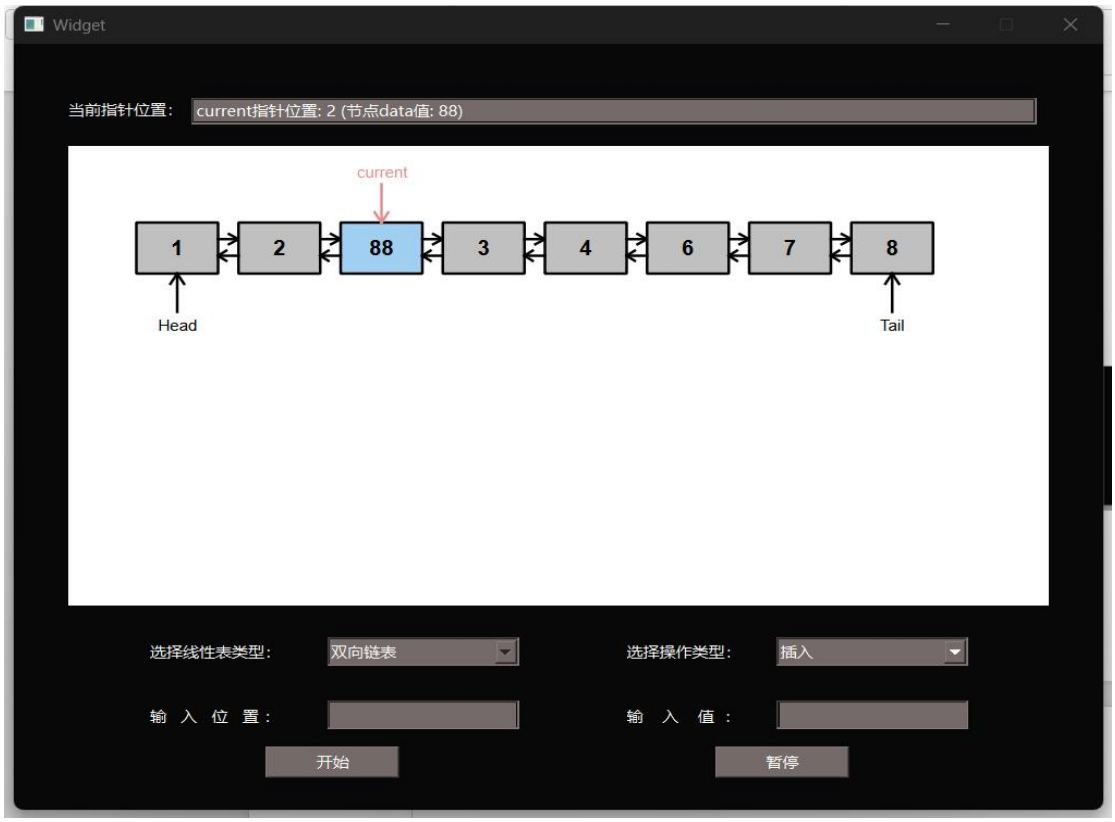

图 13：插入结果

测试数据九：循环链表 建立 “0 1 2 3 4”

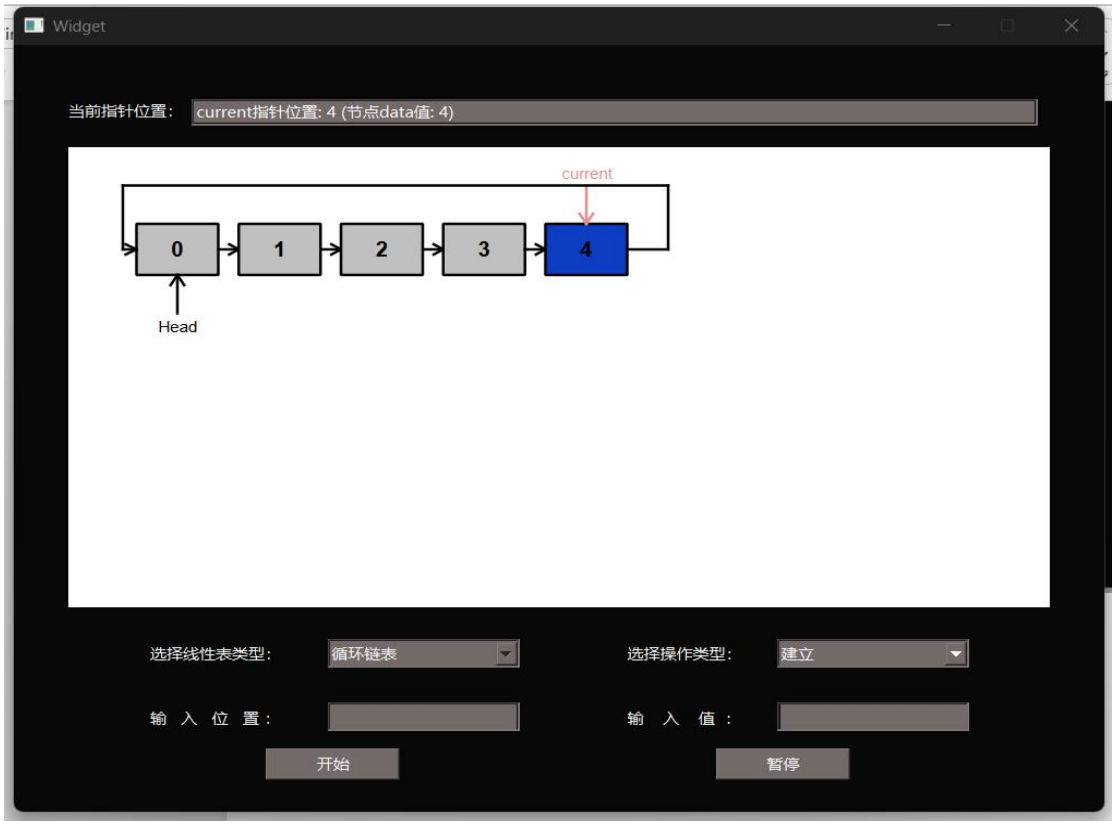

图 14：建立结果

测试数据十：循环链表 插入 “1” “66”

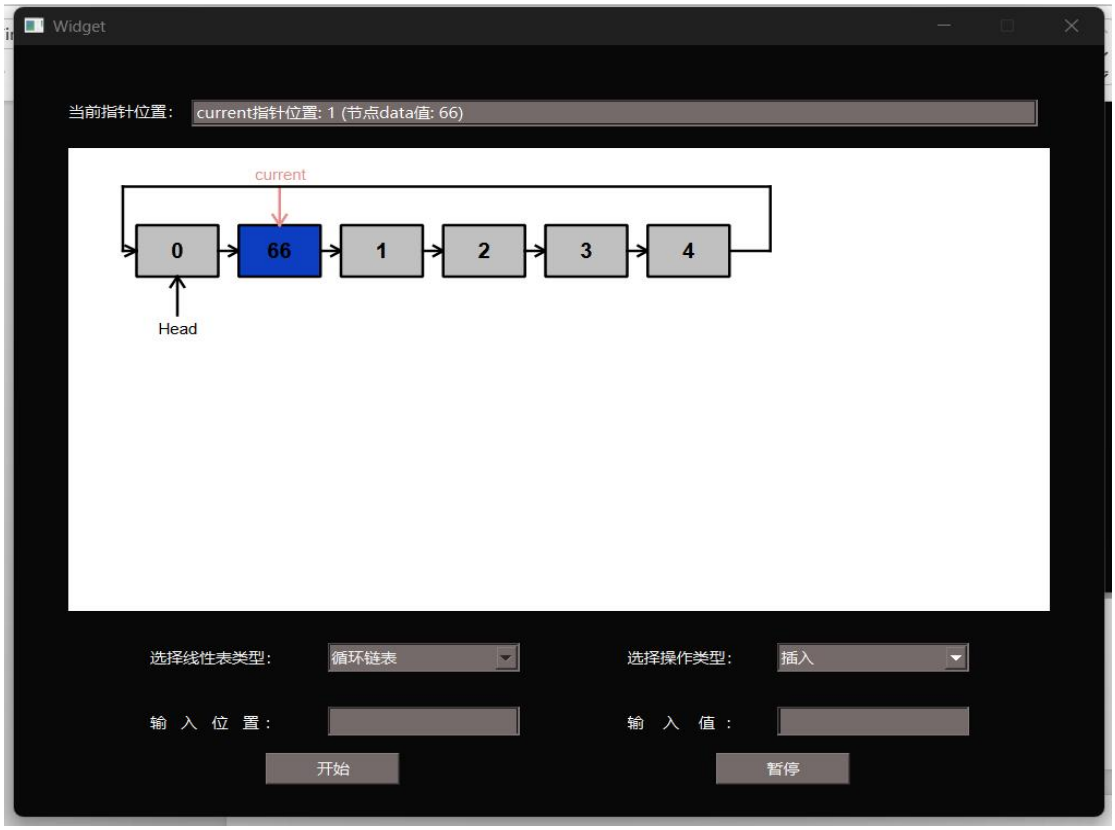

图 15：插入结果

测试数据十一：循环链表 删除 “3”

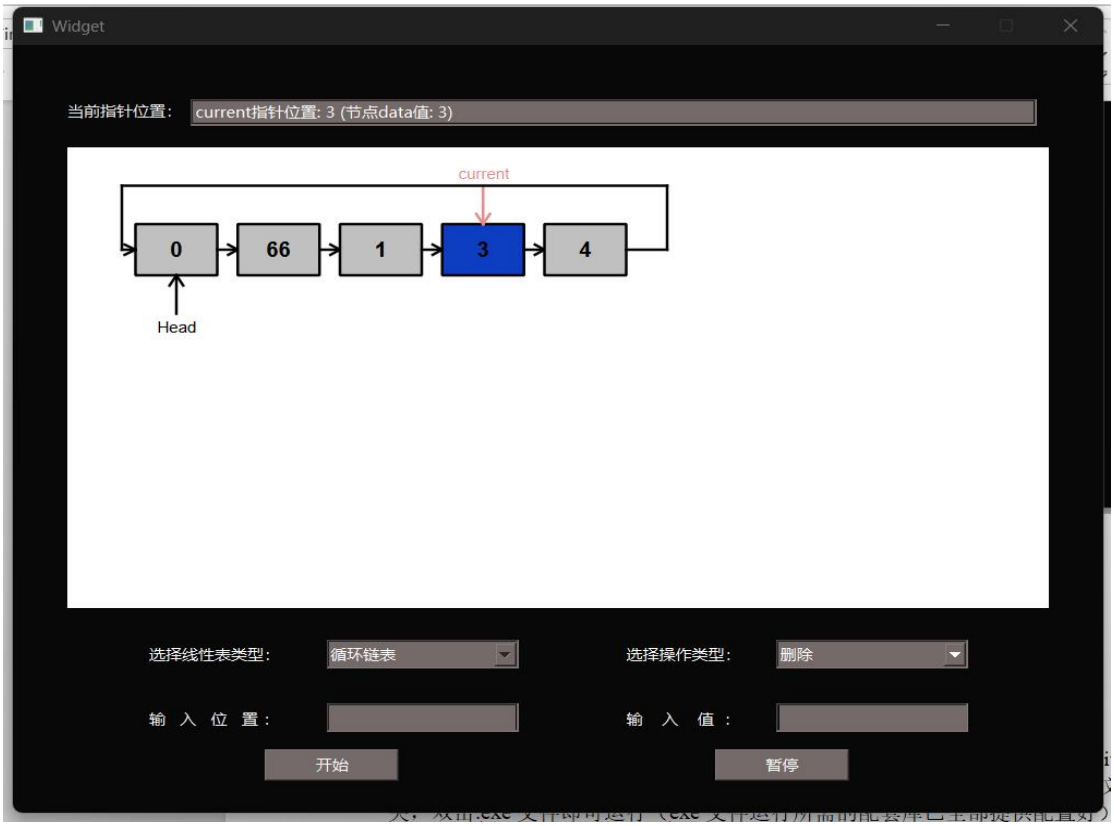

图 16：删除结果

测试数据十二：循环链表 查找 “1”

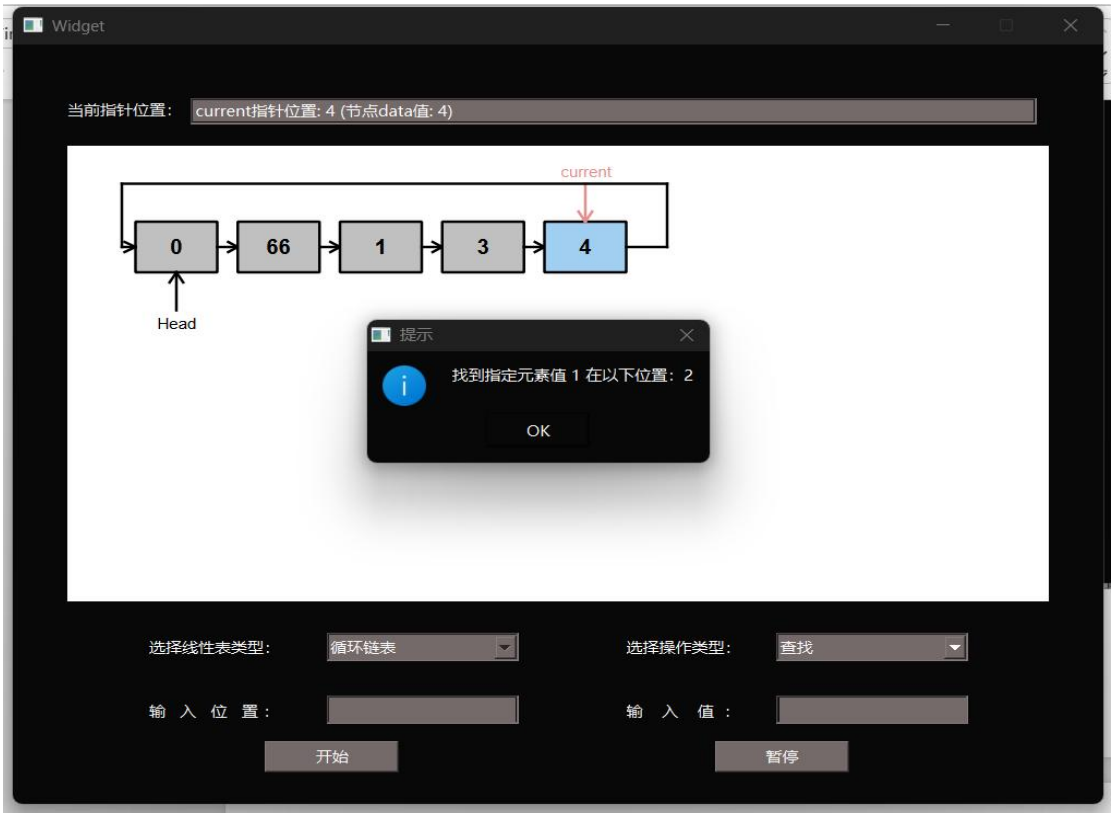

图 17：查找结果

上述运行过程中可以看出达到了该题目的设计目标。每个操作都有相应的动画演示，并且根据测试数据的结果，也可以验证任何操作的结果都是正确的，同时经过 5 位好友的功能测试，各项功能运行稳定，对于可能出现的异常/错误情况，本软件也都有较好的处理结果（弹窗提示各种错误操作的错误之处）。

本软件的优势在于设置了各项错误操作的错误处理机制，界面操作简单易上手，并且经过真人测试，软件的健壮性较好。每个操作都有完整的动画演示过程，能够很好地实时展示数据结构内的变化。

但与此同时，本软件只能进行简单的动画演示，当节点过多时（超过 16 个），无法演示。并且动画过程也有一定的弊端，就是每次都从头开始遍历，响应速度较慢（为了设计逻辑的简单，每种链表都采用从头遍历，没有结合各种链表的结构特征设计更高效的遍历策略，比如双向链表可以支持双向遍历）。同时界面过于简单，仍有一定的优化空间。

1.7 安装、运行与操作

若想检验源码正确性，自己产生可执行文件，则使用 MinGW 13.1.0 64-bit 进行编译即可；若想直接检验软件效果，则直接解压打开提交的可执行程序的文件夹，双击.exe 文件即可运行（exe 文件运行所需的配套库已全部提供配置好）。

打开 exe 文件运行程序后，即可按照“1.5 软件功能”和“1.6 系统的运行结果分析说明”中的说明进行操作。

platforms	2025/8/21 16:35	文件夹	
libgcc_s_seh-1.dll	2023/5/25 5:06	应用程序扩展	107 KB
libstdc++-6.dll	2023/5/25 5:06	应用程序扩展	2,191 KB
libwinpthread-1.dll	2023/5/25 5:06	应用程序扩展	52 KB
List_data_structure.exe	2025/8/21 20:42	应用程序	2,628 KB
Qt6Core.dll	2025/5/28 23:27	应用程序扩展	10,581 KB
Qt6Gui.dll	2025/5/28 23:27	应用程序扩展	11,063 KB
Qt6Network.dll	2025/5/28 23:27	应用程序扩展	1,988 KB
Qt6Widgets.dll	2025/5/28 23:27	应用程序扩展	6,944 KB

图 18：提交的可执行文件一览

第二部分 “综合应用”设计说明

2.1 题目

- 医院叫号系统

某医院普通门诊拥有科室 15 个，每个科室值班医生 3 人;B 超室 1 个，有 3 台 B 超机。现需设计一个叫号系统，通过挂号次序，安排每个科室叫号顺序，各科室在诊断完病人后，根据其是否开具 B 超检查要求，安排 B 超室的叫号顺序，并在所有过程中，记录每位医生所看的病人及其听诊时间。

(1)需实现挂号过程，医生就诊过程

(2)显示各科室、B 超室目前排队情况。

(3)查看各医生就诊的所有病人。

2.2 设计思想

本医院叫号/挂号系统软件开发在 Qt Creator 平台下完成，设计思想依旧分为界面开发和逻辑功能开发两大部分。界面开发注重可视化用户交互和信息展示，逻辑功能开发侧重于具体的功能实现。此设计思想和前一个“线性表操作可视化”的软件类似，在此不再赘述相同之处。不同的点在于，本挂号系统采用的是多窗口，这与单个窗口的交互略有不同，需要设计多个窗口的界面。

在具体的设计开发过程中，以各个窗口需要实现的功能为引导，规划用户交互界面的设计；通过“信号与槽”机制，提供交互界面和逻辑功能的接口；采用面向对象的编程方式，分别编写多窗口交互模块、操作实现模块、动画可视化模块等，分类实现各个操作。

2.3 逻辑结构与物理结构

本软件基于线性表这一基本数据结构进行开发设计，用于管理各科室和 B 超室的患者排队顺序（先进先出）和医生所接诊患者的信息记录。每个患者的信息用一个结构体来表示，主要保存病人的姓名、身份证号、排队号、挂诊科室、挂诊医生、听诊用时等信息。

在具体的物理结构实现方面，依赖 C++的内存管理机制 new/delete 进行链表节点的物理结构的实现。由 C++自定义类和 STL 容器等进行管理：患者节点之间通过指针连接排队顺序。数据在内存中存储并非物理连续，通过 STL 容器实现不同科室不同医生下的分组挂号管理。

2.4 开发平台

本次课程设计具体的开发平台为 Windows11 的操作系统 + Qt Creator 17.0.0(Community)的集成开发环境 + C++的编程语言(C++11 标准以上)。具体生成可执行程序的编译器为 MinGW 13.1.0 64-bit，Qt 版本 6.9.1。

2.5 软件功能

图 19：主窗口界面

选择对应的身份，点击“进入”，即可进入对应的操作端。

- 患者端

图 20：患者端窗口

1. 患者可选择科室和该科室下的医生，查看当前该科室该医生的排队情况。点击“查看”即可查看选定的科室、医生下的排队情况。【可查看 B 超室挂号情况】

2. 患者可以通过填写“姓名”和“身份证号”、“挂号科室”、“挂号医生”实现挂号过程。简化设计，“身份证号”必须是 8 位纯数字，“姓名”没有输入限制。挂号过程设计了对应的弹窗错误情况处理：【不可挂 B 超室，因为这个由医生决定。】

“姓名”输入为空时："患者姓名不能为空！"

“身份证号”输入为空时："身份证号不能为空！"

“身份证号”输入不是 8 位纯数字，含有其他字符时："身份证号格式不正确！应为 8 位数字。"

核查到挂号的“身份证号”已经挂号时："患者 xx 已挂号（xx 科室），是否取消原来挂号？"，可选择取消挂号与重新输入。

输入的“身份证号”已挂过号且与已挂号患者姓名不一致时--"请检查是否输入了正确的身份证号码！"

输入无误挂号成功时："挂号成功！患者：xx 科室：xx 医生：xx，再次点击“查看”即更新排队信息。"

不可重复挂号--"该身份证号已存在未完成的挂号记录！"

挂号后重新点击“查看”，即可查看到挂号后更新的排队情况。

3.点击“返回”，可以返回主窗口，重新选择身份，进入不同身份窗口。【实际中，选择医生身份后，应该设置医生身份核验环节，但是涉及数据库内容，故没有设计。】

- 医生端

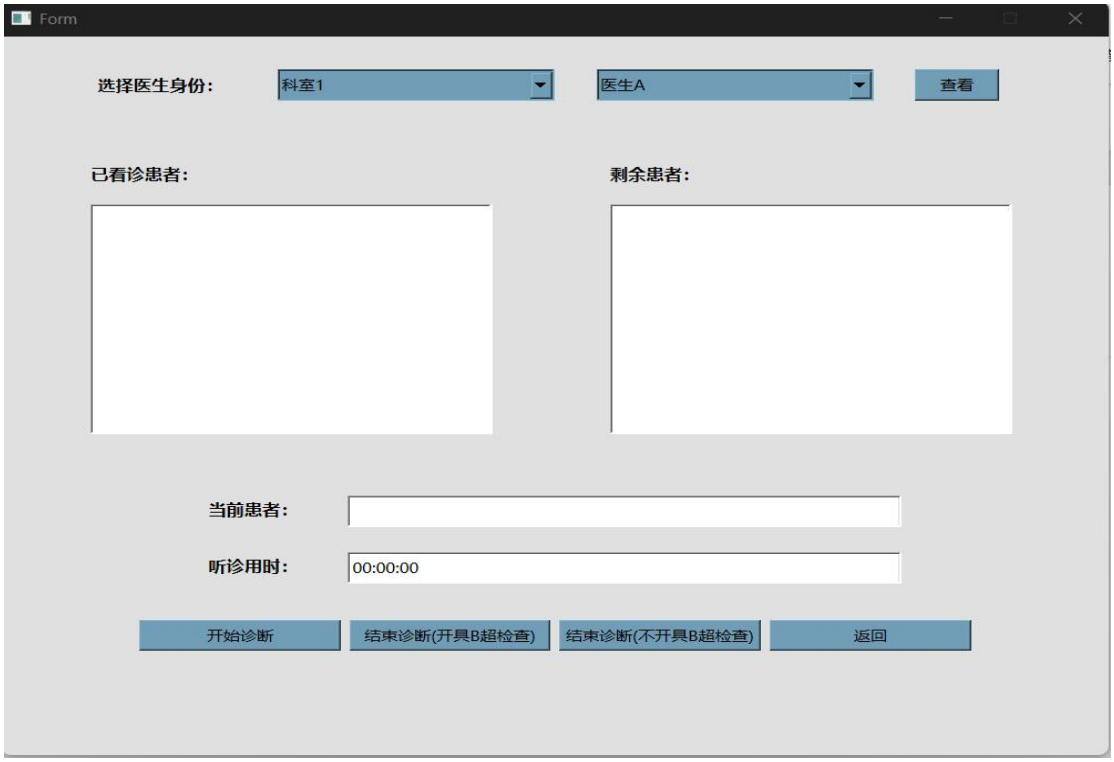

图 21：医生端窗口

1. 医生根据自己身份选择好科室和医生身份后，点击“查看”，即可查看“已看诊患者”信息（姓名、身份证号、听诊用时）和“剩余患者”信息（姓名、身份证号）
2. 点击“开始诊断”，即开始计时。“听诊用时”会实时显示用时（1 秒更新一次），“当前患者”即显示当前正在看诊的患者信息（姓名、身份证号）。
3. 结束诊断时，医生可根据患者情况选择“开具 B 超检查”和“不开具 B 超检查”。选择“开具 B 超检查”后，系统会为患者随机挂 B 超室的三台机器的任意一台，并弹窗显示；结束诊断后，会更新“已看诊患者”信息（姓名、身份证号、听诊用时）、“剩余患者”信息（姓名、身份证号）和“当前患者”信息（姓名、身份证号）。

点击“返回”，可以重新选择身份，进入不同身份窗口。【实际中，选择医生身份后，应该设置医生身份核验环节，但是涉及数据库内容，故没有设计。】

2.6 系统的运行结果分析说明

本系统在 Windows 11 操作系统，Qt 6.9.1 平台下运行，通过 MinGW 13.1.0 编译，通过 Qt Creator 17.0.0(Community)进行调试。实际运行过程中，用户可成功完成：记录每位医生所看的病人及其听诊时间、患者挂号过程、医生就诊过程、显示各科室、B 超室目前排队情况、查看各医生就诊的所有病人等功能。系统界面如图 19、20、21 所示。

接下来，将通过多组正常的测试数据，系统地展示软件各功能的运行过程及结果分析说明。（各种错误输入及软件的错误响应过程在此不再演示，可由使用者自行测试。本软件邀请了 5 位朋友进行功能测试，基本处理了所有可能出现的错误输入情况。）

测试数据一：患者端挂号

从图 19 选择“患者”身份，进入患者端操作窗口。输入患者姓名、身份证号（8 位纯数字），选择要挂的科室和医生。点击确定，经过核查（主要是身份证号的合法性核查）后即可成功挂号，再次点击“查看”即可刷新当前排队情况。

图 22：挂号完成

测试数据二：患者端查看排队情况

当前已经挂了几个患者，来验证排队情况查看功能。进入患者端操作窗口后，选择想要查看的科室、医生，点击“查看”，即可查看当前该科室该医生的排队情况。

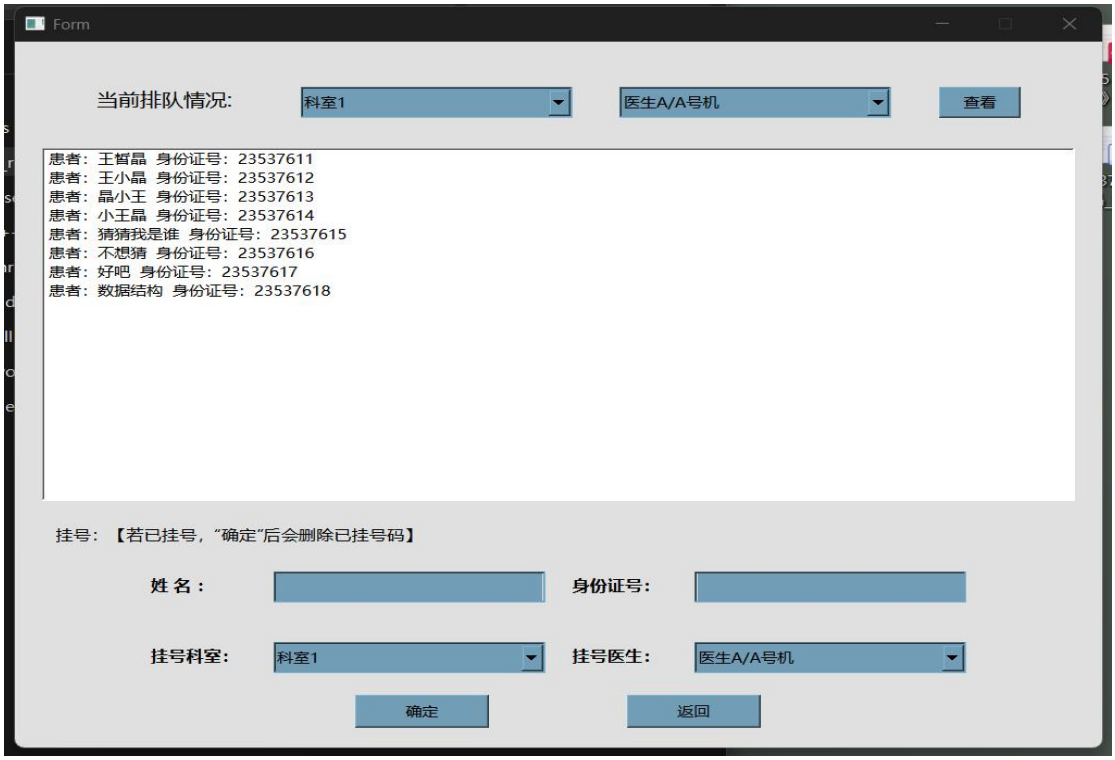

图 23：查看排队情况

测试数据三：医生端完成查看自己已看诊患者、剩余患者情况和就诊过程

从图 19 选择“医生”身份，进入医生端操作窗口。（如果当前在患者端的话，点击“返回”按钮，即可回到图 19 的主窗口。）选择好自己的医生身份后，点击查看，即可查看自己已看诊患者、剩余患者情况。点击“开始”，开始计时听诊用时。结束诊断时，选择开具 B 超诊断，则自动为患者安排 B 超诊室。选择不开具 B 超诊断，则完成诊断。点击“返回”，可返回到图 19 的主窗口。

图 24：医生端查看自己已看诊患者、剩余患者情况

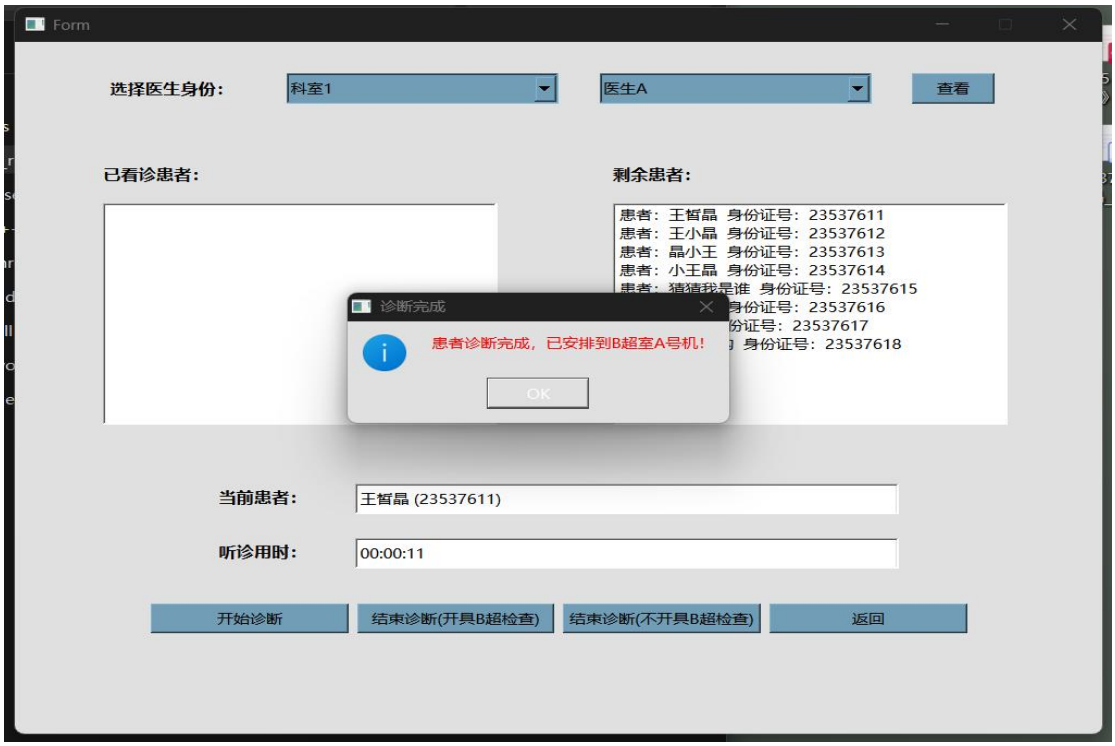

图 25：医生端选择开具 B 超诊断并结束诊断

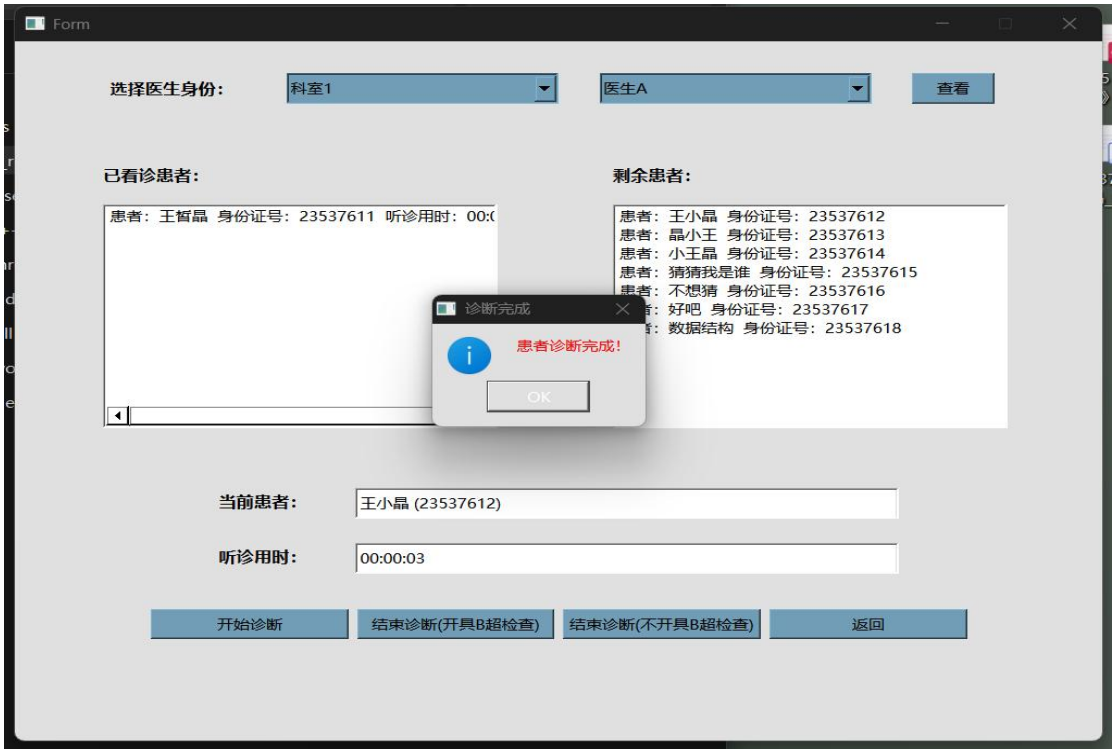

图 26：医生端选择不开具 B 超诊断并结束诊断

医生端和患者端的操作结果会同步。比如医生端完成患者就诊后，在患者端再次查看排队情况的话，也会更新为当前的真实排队情况。

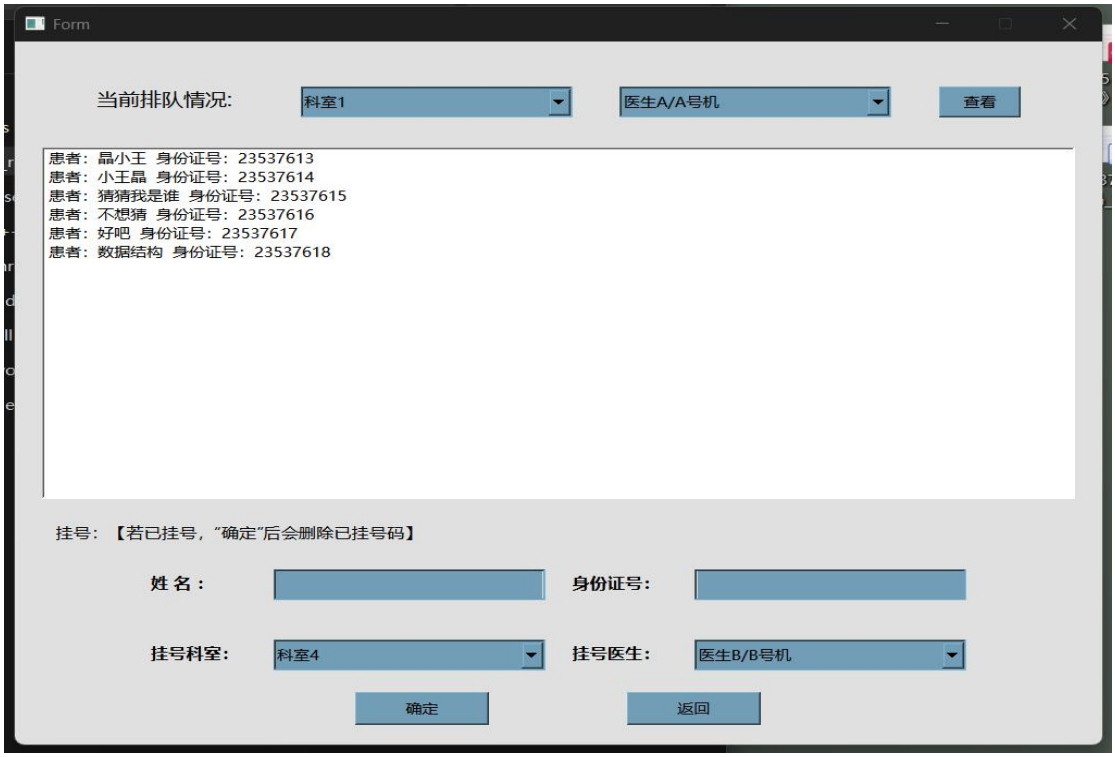

图 27：患者端同步排队情况

上述运行过程中可以看出达到了该题目的设计目标。每个身份下的各种操作都进行了实现，并且根据测试数据的结果，也可以验证任何操作的结果都是正确的，同时经过 5 位好友的功能测试，各项功能运行稳定，对于可能出现的异常/错误情况，本软件也都有较好的处理结果（弹窗提示各种错误操作的错误之处）。

本软件的优势在于设置了各项错误操作的错误处理机制和利用窗口间的交互实现不同身份的切换，界面操作简单易上手，并且经过真人测试，软件的健壮性较好。当然本软件还有可以提高的地方，比如界面设计可以更加美观、可以链接数据库进行医生身份的核验、医生 B 超室的挂号也可根据当前啊排队情况选择排队人数较少的机器进行挂号（当前 B 超机的挂号是随机的）等等。

2.7 安装、运行与操作

若想检验源码正确性，自己产生可执行文件，则使用 MinGW 13.1.0 64-bit 进行编译即可；若想直接检验软件效果，则直接解压打开提交的可执行程序的文件夹，双击.exe 文件即可运行（exe 文件运行所需的配套库已全部提供配置好）。

打开 exe 文件运行程序后，即可按照“2.5 软件功能”和“2.6 系统的运行结果分析说明”中的说明进行操作。

名称	修改日期	类型	大小
platforms	2025/8/21 16:35	文件夹	
Hospital_registration_system.exe	2025/8/22 21:27	应用程序	4,068 KB
libgcc_s_seh-1.dll	2023/5/25 5:06	应用程序扩展	107 KB
libstdc++-6.dll	2023/5/25 5:06	应用程序扩展	2,191 KB
libwinpthread-1.dll	2023/5/25 5:06	应用程序扩展	52 KB
Qt6Core.dll	2025/5/28 23:27	应用程序扩展	10,581 KB
Qt6Gui.dll	2025/5/28 23:27	应用程序扩展	11,063 KB
Qt6Network.dll	2025/5/28 23:27	应用程序扩展	1,988 KB
Qt6Widgets.dll	2025/5/28 23:27	应用程序扩展	6,944 KB

图 28：提交的可执行文件一览

第三部分 总结与收获

完成本次课程设计任务的过程中，我积累了丰富的编程实践经验，也得到了不少的收获。

在实践过程中，我先是以任务为导向，复习了线性表的相关数据结构知识和 C++的结构体、类等相关知识，之后根据课上学长的推荐，选择 Qt 进行交互界面的开发，并从零开始学习如何使用这个交互界面编程工具。一开始，我以为标准图形化交互界面的编程会非常复杂，因为在高程课上接触的伪图形化交互界面的开发确实非常复杂，需要实时追踪鼠标的位置等等，但是了解后发现 Qt 的开发机制非常容易上手。因为 Qt 的开发是非常严格地区分了界面开发和逻辑开发，而且界面设计在提供的.ui 设计师界面，直接拖动、组合相关控件即可完成界面的设计，然后建立控件和后端逻辑的信号连接就可以只去专注后端逻辑的具体实现了（这当然得益于 Qt 的信号与槽机制）。

另外，在本次项目开发的过程中，我深刻体会到了面向对象编程的魅力。面向对象的良好的封装性对于提高代码的复用性等至关重要，并且相信这次项目经历也可以帮助我更好地适应下学期的面向对象编程课程（转专业补课）。

整个项目设计开发的过程中，我独立地从需求分析、功能设计、模块组合到编码实现、测试维护几个步骤，完成了我的项目开发。并邀请我的朋友们参与我的小软件功能测试以更好地完善我的小软件。这个过程强化了我的项目工程能力和思维，我相信这对我未来的职业发展将会有不小的作用。

希望这次课程设计中的经验和收获可以帮助我更好地适应未来的学习和职业发展！

第四部分 学习资料

[1] 【QT 快速入门 | 最简单最简洁的 QT 入门教程 | 嵌入式 UI】：
https://www.bilibili.com/video/BV1N34y1H7x7?p=12&vd_source=b771feed7a3939c0a3130568bee8e248
(非常之推荐这个教程！Qt 原理讲解的很清楚，并且结合具体的小的开发实例进行讲解，能够非常快速上手 Qt 开发，对于完成此次课程设计非常有用！)